

Architecture: Order Data Platform

(Ingestion · Real-Time · Batch ETL · SQL Analytics)

The full platform built with Terraform — what each branch does and how to explain it.

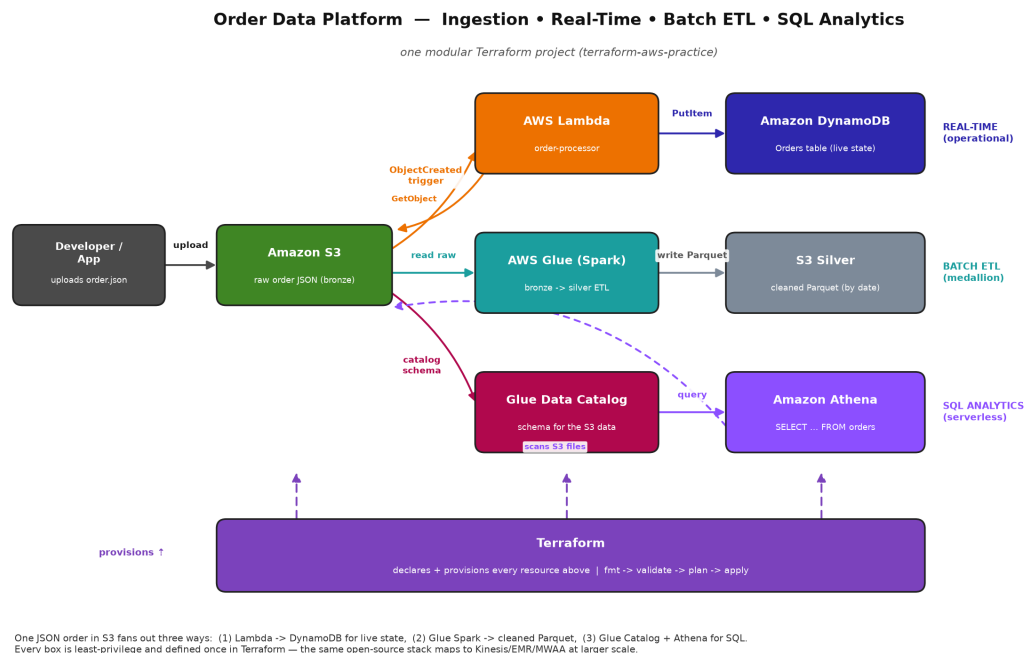


Figure 1 — The order data platform. One JSON order lands in S3 and fans out three ways: a real-time path (Lambda → DynamoDB), a batch ETL path (Glue Spark → Silver Parquet), and a SQL analytics path (Glue Catalog + Athena). Everything is provisioned by Terraform.

1. The Big Picture

This is a complete (if small) data platform. A single source of data — order JSON files landing in S3 — is used three different ways, which is exactly how real platforms separate concerns. S3 is the central hub; from there, three independent branches each serve a different need:

- **Real-time (operational):** Lambda reacts instantly and writes to DynamoDB, so the live application always has current order state (measured in seconds).
- **Batch ETL (medallion):** A Glue Spark job cleans the raw JSON into partitioned Parquet on a schedule — the bronze-to-silver medallion step that prepares data for analysis (measured in completeness).

- **SQL analytics (serverless):** The Glue Data Catalog describes the S3 data as a table, and Athena queries it with plain SQL — serverless, pay-per-query analytics.

Interview line: *"I built a platform where one source of order data fans out three ways — real-time to DynamoDB, batch ETL to cleaned Parquet, and SQL analytics through Athena — the same operational-vs-analytical split real data platforms have, all defined once as Terraform."*

2. Real-Time Branch (S3 → Lambda → DynamoDB)

1. **Upload:** A JSON order file is uploaded to S3.
 2. **Trigger:** S3's ObjectCreated event immediately triggers the order-processor Lambda — no polling, no idle server.
 3. **Process:** The Lambda calls s3:GetObject to read the file (the event only carries the bucket + key, not the data), parses it, and writes the order to DynamoDB with PutItem.
- **Result:** DynamoDB holds live order state with millisecond latency — what the app reads and writes constantly. This is the OLTP (operational) side.

3. Batch ETL Branch (S3 → Glue Spark → Silver Parquet)

4. **Read bronze:** A Glue (Spark) job reads the same raw JSON from S3 (the bronze layer).
 5. **Transform:** It deduplicates on order_id, enforces types, adds an ingestion timestamp, and drops bad rows — cleaning and conforming the data.
 6. **Write silver:** It writes the result as columnar Parquet, partitioned by order_date, to the Silver location (the data lake bucket).
- **Why it matters:** This is the bronze-to-silver medallion pattern. Parquet + partitioning means downstream queries scan far less data — the biggest performance/cost lever in a lake. This is the distributed-processing backbone of batch data engineering.

4. SQL Analytics Branch (S3 → Glue Catalog + Athena)

7. **Catalog:** The Glue Data Catalog stores the schema (columns, types, S3 location, JSON format) — Athena has no storage of its own, so this is what tells it how to read the files.
 8. **Query:** Athena runs serverless SQL directly against the S3 files using that schema, writing results to a dedicated results bucket.
- **Result:** Analysts answer questions like 'revenue by restaurant' with SELECT restaurant, SUM(amount) FROM orders GROUP BY restaurant — no servers, pay per TB scanned. This is the OLAP (analytical) side, complementing DynamoDB's operational side.

5. The Components (Quick Reference)

- **Amazon S3:** Object store + entry point; the central hub every branch reads from. Holds the raw order files (bronze).
- **AWS Lambda:** Serverless compute; reacts to each new file and writes to DynamoDB. The real-time worker.
- **Amazon DynamoDB:** NoSQL database; live order state for the app. Operational/OLTP.
- **AWS Glue (Spark):** Managed Spark; the batch ETL engine that cleans bronze into silver Parquet.
- **Glue Data Catalog:** Metadata/schema layer that makes S3 files queryable as tables.
- **Amazon Athena:** Serverless SQL engine over S3. Analytical/OLAP.
- **Terraform:** Infrastructure as Code; declares and provisions every box once, reproducibly. The blueprint.

6. How This Maps to Production Scale

The same shape scales up by swapping components for their larger-scale cousins — the pattern is identical:

- S3 stays S3 (the universal data lake).
- Lambda → still Lambda, or Kinesis + a stream processor when data arrives as a continuous stream instead of files.
- Glue Spark → Glue at larger scale, or EMR for very heavy/custom Spark workloads.
- Athena → Athena, or Redshift when you need a dedicated high-performance warehouse.
- Terraform → still Terraform; orchestration by Step Functions or MWAA (managed Airflow) as pipelines grow.

7. The One-Paragraph Interview Pitch

Interview line: *"I built an order data platform on AWS, entirely as Terraform. Order JSON lands in S3 and fans out three ways: S3's event triggers a Lambda that writes to DynamoDB for real-time state; a Glue Spark job transforms the raw JSON into cleaned Parquet partitioned by date — the bronze-to-silver medallion pattern; and the Glue Data Catalog plus Athena let analysts query the data with SQL, serverless. Every service uses a least-privilege IAM role, and the whole platform is one modular Terraform project I can reproduce or tear down with a single command. It covers the four things a data-engineer role asks about — ingestion, real-time processing, batch ETL, and SQL analytics."*